SOFTWARE REQUIREMENTS SPECIFICATION (SRS) FOR RESTAURANT APP MVP

1. Introduction

1.1 Purpose: This document defines the requirements of the Restaurant App MVP, its intended audience (Diners and Managers), and the full-stack architecture featuring a Node.js/Express backend and MongoDB database for Fall 2026.

1.2 Scope:

In Scope: User authentication (JWT), role-based access control, Pakistani Menu browsing, search with debouncing, global themes, cart management, custom hooks, table reservation, Manager dashboard, Node.js/Express backend server, and MongoDB database integration.

Out of Scope: Real online payments and push notifications.

Resolved App Limitations & Solutions:

Problem: Data loss on app reinstallation and isolated phone storage where devices do not share data. *Solution:* Implemented a centralized MongoDB database via an Express backend so all phones read and write to a single source of truth.

Problem: A manager updating a dish price or marking it unavailable on their phone did not reflect on customers' phones. *Solution:* Centralized menu state on the server, allowing instant reflection of price and availability updates across all client devices upon refresh or polling.

1.3 Definitions and Acronyms

SRS: Software Requirements Specification

MVP: Minimum Viable Product

API (Application Programming Interface): A set of functions and protocols allowing different software applications to communicate.

REST (Representational State Transfer): An architectural style for designing networked applications using HTTP requests.

Endpoint: A specific URL where an API can access the resources it needs from a server.

JSON (JavaScript Object Notation): A lightweight data-interchange format easily read and written by humans and machines.

MongoDB: A NoSQL document-oriented database used for storing data in flexible, JSON-like collections.

Collection: A grouping of MongoDB documents (equivalent to a table in relational databases).

Mongoose: An ODM (Object Data Modeling) library for MongoDB and Node.js that manages data relationships and validation.

Middleware: Functions executed during the lifecycle of an HTTP request to the Express server (e.g., authentication checks).

JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties for secure authentication.

HTTP Status Code: Standardized codes signaling the outcome of an HTTP request (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error).

2. Overall Description

2.1 User Roles:

Customer: Browse menu, manage cart, apply promo codes, book tables, place and track orders.

Manager: Manage incoming orders, accept/decline table reservations, update menu item availability and prices via protected endpoints.

2.2 User Stories:

As a customer, I want to browse Pakistani dishes so that I can order food quickly.

As a customer, I want to filter items by category so that I can find my favorite Biryani or Karahi.

As a customer, I want to view my order tracking status in real-time via polling.

As a manager, I want to view incoming orders so that I can process them efficiently.

As a manager, I want to toggle menu item availability so that unavailable dishes are hidden from customers.

3. Functional Requirements

FR-01: The system shall allow users to log in or sign up with role selection (Customer/Manager) with secure password hashing.

FR-02: The system shall display a Pakistani menu fetched dynamically from the database, categorized into Starters, Mains, Desserts, and Drinks.

FR-03: The system shall provide a debounced search bar to filter menu items from the server.

FR-04: The system shall manage shopping cart state using useReducer supporting item increments, decrements, and notes.

FR-05: The system shall allow table reservations with backend validation ensuring tables are free and have sufficient seats.

4. Non-Functional Requirements

Usability: Simple, intuitive interface tailored for Pakistani users.

Performance: Optimized client-side re-renders with server-backed response times under 500ms.

Security: Password encryption using bcryptjs and route protection using JWT authorization headers.

5. Database Design (MongoDB Collections)

Collection	Fields and Types	Relationships and Rules
Users	name (String, required), email (String, required, unique), password (String, required), role (Enum: ['customer', 'manager'], default: 'customer')	Independent collection; serves as authentication reference for tokens.
MenuItems	name (String, required), description (String), price (Number, required, min: 0), category (String, required), image (String), isAvailable (Boolean, default: true)	Independent collection; managed exclusively by users with the manager role.
Tables	tableNumber (Number, required, unique), capacity (Number,	Referenced by Reservations to verify

Collection	Fields and Types	Relationships and Rules
	required, min: 1), isReserved (Boolean, default: false)	physical seating availability.
Reservations	user (ObjectId, ref: 'User', required), table (ObjectId, ref: 'Table', required), date (String, required), time (String, required), guests (Number, required), status (Enum: ['Pending', 'Confirmed', 'Cancelled'], default: 'Pending')	Links a User to a specific Table with date/time validation rules to prevent double booking.
Orders	user (ObjectId, ref: 'User', required), items (Array of objects containing menuItem ref, quantity, and price), totalAmount (Number, required), status (Enum: ['Pending', 'Preparing', 'Ready', 'Served', 'Cancelled'], default: 'Pending')	Links to User and MenuItems; total amount is calculated server-side based on database prices.

6. System Architecture & Deployment

Client-Server Architecture: React Native mobile app communicates via REST APIs with an Express.js backend connected to MongoDB Atlas.

7. Diagrams

Sequence Diagram ("Customer places an order"):

Lifelines: Mobile App $\rightarrow$ Express Server $\rightarrow$ MongoDB.

Flow: App sends POST /api/orders with token and items $\rightarrow$ Server validates token via middleware, queries MongoDB for item prices, computes total amount, creates order document in database $\rightarrow$ Database returns saved order $\rightarrow$ Server responds to App with 201 Created and order details.

Deployment Diagram:

Nodes: Mobile Phone (running React Native App via Expo Go) $\longleftrightarrow$ Laptop (running Node.js/Express server on local IP port 5000) $\longleftrightarrow$ MongoDB Atlas Cloud Database.

8. API Endpoints

Method	Endpoint	Access	Request Body	Response
POST	/api/auth/register	Public	name, email, password, role	token and user details
POST	/api/auth/login	Public	email, password	token and user details
GET	/api/menu	Public	None	List of all menu items
GET	/api/menu/:id	Public	None	Single menu item object
POST	/api/menu	Protected (Manager)	name, description, price, category	Created menu item (201)
PUT	/api/menu/:id	Protected (Manager)	name, price, isAvailable, etc.	Updated menu item object
DELETE	/api/menu/:id	Protected (Manager)	None	Success message
POST	/api/orders	Protected (Customer)	items array, delivery notes	Created order object (201)
GET	/api/orders/my	Protected (Customer)	None	List of logged-in customer's orders
GET	/api/orders	Protected (Manager)	None	List of all system orders
PATCH	/api/orders/:id/status	Protected (Manager)	status	Updated order status object

Method	Endpoint	Access	Request Body	Response
POST	/api/reservations	Protected (Customer)	tableId, date, time, guests	Created reservation object (201)
GET	/api/reservations/my	Protected (Customer)	None	List of customer's table bookings
GET	/api/reservations	Protected (Manager)	None	List of all table reservations
PATCH	/api/reservations/:id	Protected (Manager)	status	